

PageIndex Implementation Flow

document → page text → ToC detection → tree construction → index verification → retrieval-ready document record

1. **Parse pages:** Extract plain text from the pdf.
2. **Detect existing ToC:** Decide whether the document already contain a ToC.
3. **Choose tree-building path:**
 - **ToC w/ page numbers:** Converts to **JSON** with page numbers.
 - **ToC w/o page numbers:** LLM identifies where each section starts.
 - **If no ToC exists:** LLM infers the hierarchy directly from page-marked text.
4. **Group pages:** Long documents are split into token-safe groups before prompting. This keeps each LLM call within context limits and avoids forcing the model to read the whole document at once.
5. **Verify and repair indexes:** Generated section boundaries are checked against the document text. If a section appears to start on the wrong page, the system searches nearby pages and asks the LLM to repair the boundary.
6. **Convert section list to tree:** The cleaned section list is converted into a recursive hierarchy. Each node receives a **start_index**, an **end_index**, and child nodes when the section contains smaller sections.
7. **Split large nodes:** Sections that are too large are processed again at a finer level, creating deeper subtrees only where the document needs more detail.
8. **Add retrieval fields:** The final tree can include **node_id**, raw **text**, node **summary**, and a document-level description depending on the indexing settings.
9. **Store retrieval-ready data:** The system stores the document record, tree, page count, extracted page text, and optional descriptions so later retrieval can return either the compact tree or the raw evidence.
10. **Support query-time retrieval:** At query time, the system can expose document meta-data, a tree without raw text, and selected raw page evidence for answer generation.

LLM usage note: These steps can use the same model, but they are separate LLM calls. The pipeline passes explicit data between steps, such as extracted page text, ToC JSON, selected **doc_id**, selected **node_id**, and retrieved evidence. It does not rely on one shared hidden chat history.

Indexing Step 1: Create PageIndex Tree for Each Document Using LLM

- **Input:** Document
- **Output:** JSON-based hierarchical structure that represents ToC
- **LLM role:** The LLM can be used to identify section boundaries, generate titles, write summaries, and decide how detailed the tree should be.

Implementation detail:

1. **Extract page text:** Extract ordered page-level text from document.

- The actual extraction output is a list of (page_text, token_count) pairs.
- The physical page index is implied by the list position: the first pair is page 1, the second pair is page 2, etc.

```
[  
  ("Annual Report 2023 ...", 742),  
  ("Financial Stability ...", 691)  
]
```

- Before the LLM receives the page text, the ordered pages are wrapped with physical page tags:

```
<physical_index_1>  
Annual Report 2023 ...  
</physical_index_1>  
  
<physical_index_2>  
Financial Stability ...  
</physical_index_2>
```

2. **Detect existing ToC using LLM:**

- Using LLM to label whether each page is a ToC. Since existing structure usually reflects hierarchy structure.

```
toc_page_list = [1, 2]
```

- Page-check behavior:

Situation	Pages checked
No ToC found	Up to ~20 pages (PDF pages 1–20), then stop
ToC found early	Only until the ToC block ends (could be 3 pages, could be 8)
ToC spans 20 pages	More than 20 – keeps going while still inside the ToC block

3. **Detect existing ToC using LLM:**

4. Concatenate those pages' text and feed to LLM to detect whether page numbers are given for sections in the ToC.

Example input to that one call:

```
Contents  
1. Introduction ..... 3  
2. Monetary Policy ..... 9  
3. Financial Stability ..... 21  
  
-> page_index_given_in_toc: "yes"  
  
vs.
```

```

Contents
1. Introduction
2. Monetary Policy
3. Financial Stability

-> page_index_given_in_toc: "no"

```

5. **Build the first tree:** If a ToC exists, the system normalizes it into JSON and aligns its section pages to physical pages. If no ToC exists, the LLM reads page-marked text and proposes section titles, hierarchy, and start pages.
6. **Handle long documents:** The document can be divided into token-safe groups. The LLM reads one group at a time and returns partial structure, then the system merges the partial structure into one document tree.
7. **Verify page boundaries:** Section titles and start pages are checked against the extracted text. When a boundary is wrong, the system searches nearby pages and repairs the index.
8. **Create recursive nodes:** The final structure is converted into nested nodes. Parent nodes cover broad sections; child nodes cover more specific subsections.
9. **Control detail level:** The tree becomes deeper when a section is too long or too dense. Short sections can remain as leaf nodes to avoid unnecessary complexity.
10. **Attach retrieval fields:** Each node receives a stable **node_id**, page range, optional summary, and optional raw text pointer.

- **Nodes** represent a section (e.g. chapter, paragraph, page)

```

Node {
  node_id: string,           // Unique node identifier
  title: string,             // Human-readable label or title
  description: string,       // Optional detailed explanation of the node
  metadata: object,          // Arbitrary key-value pairs for context or attributes
  sub_nodes: [Node]         // Array of child nodes (recursive structure)
}

```

- **node_id:** Serves as a reference key to locate the corresponding raw data.
- **sub_nodes:** Field allows recursive nesting, forming a full ToC tree.
- **metadata:** Field can store semantic information (e.g. document type, author, timestamp, or relevance scores.)

- **ToC Example:**

```

...
{
  "node_id": "0006",
  "title": "Financial Stability",
  "start_index": 21,
  "end_index": 22,
  "summary": "The Federal Reserve ...",
  "sub_nodes": [
    {
      "node_id": "0007",
      "title": "Monitoring Financial Vulnerabilities",

```

```

    "start_index": 22,
    "end_index": 28,
    "summary": "The Federal Reserve's monitoring ..."
  },
  {
    "node_id": "0008",
    "title": "Domestic and International Cooperation and Coordination",
    "start_index": 28,
    "end_index": 31,
    "summary": "In 2023, the Federal Reserve collaborated ..."
  }
]
}
...

```

- **start_index / end_index**: Start page / End page
 - Each node in ToC links to corresponding **raw content** (e.g. text, images, tables)
- node_id** → node_content (raw content, extracted text, images, etc.)
- This mapping enables the LLM to **select and retrieve** specific nodes as needed.
 - **JSON-based ToC index** resides within the LLM's active reasoning context.

Indexing Step 2: Create Documents Record Using LLM (Can be Same LLM)

The document-level record answers: “Which document should answer the query?” This record should be small, stable, and easy to filter.

- **Main fields**: **doc_id**, file name, title, source path, document type, author, date, language, and access permission.
- **Description**: A short document-level summary used for document search.
 - For a small collection, create the PageIndex ToC tree first, then derive the document-level description from the root summary, top-level node summaries, and metadata.
 - For a large collection, create a cheaper document-level record first using metadata, filename, abstract, title page, or a short extracted preview. Generate full PageIndex trees only for documents that are likely to be searched.
- **Storage pointer**: The record should point to where the full ToC tree, raw text, page images, or original PDF are stored.

Implementation detail:

1. **Create a stable document id**: Each document receives an id that can be used throughout the whole pipeline. Query-time retrieval returns this id before searching inside the document tree.
2. **Collect basic metadata**: The record stores stable facts such as file name, title, source, year, type, language, and permission. These fields are useful for filtering and access control.
3. **Generate or derive a description**: The description is a short document-level summary. It can be written by the LLM from the root and top-level tree summaries, or from a short preview when the full tree has not been created yet.

4. **Store pointers:** The record keeps locations for the original file, tree, extracted text, page images, or other raw evidence. The record itself stays small while pointing to larger stored objects.
5. **Separate document record from tree:** The document record is used to decide **which document** to search. The PageIndex tree is used later to decide **which section** inside that document to search.
6. **Keep the record updateable:** If the document is re-indexed, the record can keep the same **doc_id** while updating its description, metadata, and storage pointers.

Document-Level Record Example:

```
{
  "doc_id": "fed_annual_2023",
  "file_name": "annual-report-2023.pdf",
  "title": "Federal Reserve Annual Report 2023",
  "description": "Annual report for document-level search.",
  "metadata": {
    "source": "Federal Reserve",
    "year": 2023,
    "document_type": "annual_report",
    "language": "en"
  },
  "toc_tree_uri": "storage/trees/fed_annual_2023.json",
  "raw_file_uri": "storage/files/annual-report-2023.pdf"
}
```

doc_id → document record → ToC tree / node_map / raw files

Indexing Step 3: Build Optional Document-Search Index

This step is needed when the system has many documents. It helps choose which documents should be searched before running tree search inside them.

- **SQL metadata search:** Good for exact filters such as year, source, company, document type, or permission.
- **Vector search:** Good for semantic matching against document descriptions, titles, abstracts, or top-level summaries.

Implementation detail:

1. **Choose searchable fields:** The index can use title, description, metadata, abstract, root summary, and top-level section summaries. Raw full-document text is usually too large for document-level search.
2. **Build metadata filters:** Structured fields are stored in a database so the system can apply exact filters before semantic search. Examples include year, source, language, document type, and permission.
3. **Build semantic search:** Document descriptions and high-level summaries can be embedded into vectors. The user query is embedded at query time and matched against these document vectors.

4. **Combine retrieval signals:** Production systems often combine metadata filters, keyword search, vector similarity, and optional LLM reranking. This reduces the number of documents passed into tree search.
5. **Return candidate documents:** The output is a small ranked set of `doc_id` values. Tree search then runs only inside those selected documents.
6. **Handle scale:** For small collections, this step can be simple or skipped. For large collections, it is important because generating and searching every full tree for every query is expensive.
7. **Keep the index synchronized:** When documents are added, deleted, or re-indexed, the document-search index should update together with the document records.

Query Step 1: Document Search (Can be same LLM)

It uses the document-level index and returns candidate `doc_id` values.

- **Input:** User question, optional filters, and conversation context if needed.
- **Output:** A ranked list of candidate documents.

Query Step 2: Tree Search (Can be same LLM)

The LLM reads the PageIndex ToC tree for each selected document, usually without the full raw document text, and selects relevant `node_id` values.

- **Input:** User question, selected `doc_id`, and that document's ToC tree.
- **Output:** Relevant `node_id` values, often with short reasons.

Query Step 3: Retrieve Raw Content

After tree search selects nodes, the system uses `node_map` to fetch the real evidence.

Query Step 4: Answer the Question (C)

- **Input:** User question, retrieved evidence, page numbers, `doc_id`, and `node_id`.
- **Output:** A grounded answer with citations to pages, nodes, or documents.

Different Methods

Document Search vs. Tree Search

In a multi-document system, retrieval is usually split into two levels.

- **Document search:** Find the relevant document records. The output is one or more `doc_id` values.
- **Tree search:** Search inside each selected document's PageIndex ToC tree. The output is one or more `node_id` values.

```
query → doc-search → doc_id → tree-search → node_id → raw content → answer
```

- **Document search tutorial (doc-search):** Shows how to choose documents using meta-data, semantics, or LLM-generated descriptions.
- **Tree search tutorial (tree-search):** Shows how to choose relevant nodes inside one known document.

Manual Text RAG: [demo/PageIndex_Simple](#)

This demo starts at the **tree-search** level because the document is already known.

1. **Convert PDF to Tree(ToC):** PageIndex creates node titles, summaries, page indexes, raw text, and node ids.
2. **Tree search:** Give the LLM the question and the tree without raw text. The LLM returns relevant **node_id** values.
3. **Map nodes to content:** Use **node_map** to locate the full content for each selected id.
4. **Generate answer:** Feed only the selected node text to the LLM to answer.

Production note: For many documents, add a document-search step before this flow.

Vision RAG: [demo/Vision_PageIndex](#)

This demo also starts from one known document, but the final answer is generated from rendered page images.

1. **Convert PDF to Tree(ToC):** PageIndex creates node titles, summaries, page indexes, raw text, and node ids.
2. **Render pages:** Convert PDF pages into image files.
3. **Tree search:** Use a multimodal model as a text-only model to select nodes from the tree.
4. **Collect page images:** Convert selected nodes into page ranges and get page images.
5. **Generate visual answer:** Send the question and selected page images to a vision-language model.

Production note: Use this when figures, tables, scanned pages, or visual layout are important.

Chat Quickstart: [demo/Chat_Quickstart_PageIndex](#)

This demo hides the tree-search and answer-generation details behind the PageIndex chat API.

1. **Submit document:** Upload a PDF and keep the returned **doc_id**.
2. **Check status:** Wait until PageIndex finishes processing.
3. **Ask question:** Call `pi_client.chat_completions(...)` with the user question and **doc_id**.
4. **Stream answer:** PageIndex handles retrieval and answer generation internally.

Agentic Retrieval: demo/Agentic_PageIndex

Agentic retrieval is not just “LLM retrieves then LLM answers.” Normal PageIndex can also do that. The difference is that retrieval becomes a **tool** inside a larger agent workflow.

1. **Submit document:** Upload a PDF and keep **doc_id**.
2. **Ask retrieval prompt:** Request raw relevant evidence in a structured JSON format.
3. **Parse JSON:** Extract page numbers and evidence content from the model output.
4. **Check / reuse evidence:** An agent can validate, compare, call more tools, retrieve again, or pass evidence to another step.
5. **Generate final response if needed:** The final answer may be produced after the evidence is collected.

Key distinction:

- **Normal PageIndex:** Retrieval is a step inside a fixed QA pipeline.
- **Agentic PageIndex:** Retrieval is an iterative tool that an agent can call conditionally.

Comparison

Method	Main Role	Flow Summary
Document Search	Select document(s)	Query → document index → doc_id
Tree Search	Select section(s)	doc_id → ToC tree → node_id
Manual Text RAG	Custom text QA	Tree → node ids → node text → answer
Vision RAG	Visual document QA	Tree → node ids → page images → VLM answer
Chat Quickstart	Simple hosted QA	Document id → PageIndex chat API → streamed answer
Agentic Retrieval	Structured evidence tool	Query → retrieval prompt → JSON evidence → agent use